

第八章 数据库编程（嵌入式 SQL）—— 选择题解析

第 1 题

答案：B

解析：

A 是经典误导。游标不是 C 语言的指针，它是数据库系统提供了一种逐行访问机制。它和 C 指针唯一的共同点是名字里都有"指"字，本质完全不同。

B 正确。游标的核心特征就是只能向前，不能后退。你 FETCH 完第 5 行想回到第 3 行？不行，只能关了重新来一遍。

C 错。DECLARE 只是"声明"——告诉数据库"我要一个游标，它将来会查这个 SQL"，此时 SQL 还没执行。真正执行是在 OPEN 的时候。

D 错。CLOSE 关闭游标后，结果集就被释放了，不保留在内存中。

一句话记住：DECLARE 声明 → OPEN 执行并取出结果 → FETCH 逐行拿 → CLOSE 释放。

第 2 题

答案：A

解析：

这段代码的意图是：把 C 变量 sno 的值 ("S001") 传给 SQL，查出对应的学生姓名。

错误在第 17 行：WHERE Sno = sno。在 SQL 语句里引用 C 变量（主变量），必须加冒号前缀，写成 WHERE Sno = :sno。否则数据库会把 sno 当成一个列名，去 Student 表里找叫 sno 的列——当然找不到。

B 错：INTO 子句正是 SELECT 用来把查询结果写入主变量的，语法正确。

C 错：char sname[20] 声明没问题，姓名用字符串存很合理。

核心规则：SQL 里看 C 变量，前面必须带冒号。这是嵌入式 SQL 最容易犯的错误，没有之一。

第 3 题

答案：B

解析：

SQLCODE 是嵌入式 SQL 的状态码：

- 0 → 执行成功
- 100 → 没有更多数据了（查询结果为空，或者 FETCH 已经拿完了所有行）
- 负数 → 出错了

所以 SQLCODE = 100 不是错误，是"已经拿完了"的信号。对应的操作应该是退出循环，而不是继续 FETCH（继续 FETCH 也只会再返回 100）。

A 说"执行失败"——错，100 不是失败码。

C 说"执行成功，继续 FETCH"——错，100 意味着没数据可拿了。

D 说"权限不足"——错得离谱。

100 不是错误码，是"没货了"的信号。就像你去取快递，取完了再刷条码，机器说"没有更多包裹了"

。



第 4 题

答案：C

解析：

指示变量 (indicator variable) 是和主变量配对使用的"小跟班"，专门解决一个问题：数据库的 NULL 在 C 语言里没法表示。

它的规则：

- 0 → 主变量值非 NULL，且完整（没被截断）
- 负值（通常是 -1）→ 主变量值为 NULL
- 正值 → 主变量值被截断了（比如数据库字段是 varchar(100)，但 C 的缓冲区只给了 20 字节，多出来的被截掉了，指示变量就告诉你截了多少）

A 错：指示变量必须是 short 或 int 等整数类型，不能是任意类型。

B 错：0 表示非 NULL，不是 NULL。

C 正确：正值表示截断。

D 错：前半句正确（必须在同一个 DECLARE SECTION 中声明），但后半句错误——类型不必一致。主变量可以是 char，指示变量是 short，它们配对使用。

指示变量 = 主变量的"质检员"：0 表示合格，负值表示是 NULL，正值表示被截断了多少字节。

第 5 题

答案：D

解析：

A 正确：不连接数据库，啥也干不了。

B 正确：可以同时连多个数据库，用不同的连接名区分。

C 正确：DISCONNECT 断开后当然可以重新 CONNECT，就像挂了电话可以再打。

D 错误：连接可以切换。嵌入式 SQL 支持多连接，通过 AT 连接名 指定用哪个连接执行 SQL。程序运行期间可以在不同连接之间切换。

数据库连接就像多开浏览器标签页——可以同时开多个，也可以在不同标签页之间切换，不是"连上了就不能动"。

第 6 题

答案：B

解析：

嵌入式 SQL 中，C 语言和数据库之间传递数据靠两个东西：

- 主变量 (host variable)：传递数据。分输入和输出两种。
- SQLCA (SQL Communication Area)：传递状态信息 (SQLCODE 就在这里面)。

A 错：说反了——SQLCA 传状态，主变量传数据。

B 正确：输入主变量是 C 赋值给 SQL 用的（比如 WHERE 条件里的值），输出主变量是 SQL 把查询结果写进去返回给 C 的（比如 INTO 子句）。

C 错：游标用于传递多条记录（逐行拿），主变量用于传递单条记录。说反了。

D 错：指示变量用于表示 NULL 或截断，不是表示 SQL 是否执行成功（那是 SQLCODE 的事）。

记住分工：主变量传数据，SQLCA 传状态，指示变量给主变量打标签（是不是 NULL、有没有截断）。



第 7 题

答案：B

解析：

WHERE CURRENT OF <游标名> 是嵌入式 SQL 里的"定点更新/删除"语法——只更新/删除游标当前指向的那一行。

A 错：不能用于"任意" UPDATE，必须配合游标使用，而且 UPDATE 的目标表必须是游标查询的那个表。

B 正确：就是更新当前行。

C 错：CURRENT OF 只影响一行，就是游标当前指向的那一行。

D 错：没有游标就没有 CURRENT OF，它们是绑定关系。

WHERE CURRENT OF = "就是我现在指的这一行，别的不动"。类似你用鼠标点中一个文件然后按删除键——只删这一个。

第 8 题

答案：A

解析：

A 正确：COMMIT 提交事务后，所有和该连接相关的、未关闭的游标会被自动关闭。这是因为提交后事务结束，游标的结果集可能不再有效。

B 错：ROLLBACK 不仅撤销修改，游标也会受影响——通常游标会被关闭或变得不可用（具体取决于 DBMS 实现），不是"游标状态保持不变"。

C 错：不需要每个 FETCH 都 COMMIT。通常是一批操作完成后统一 COMMIT。

D 错：嵌入式 SQL 程序必须显式管理事务，要么 COMMIT 要么 ROLLBACK，不能指望 DBMS 自动管理。

COMMIT 像"保存游戏"——保存后之前打开的对话框（游标）就自动关了。ROLLBACK 像"读档"——回到存档点，中间的进度全没了。

第 9 题

答案：B

解析：

嵌入式 SQL 的编译是两步走：

1. 预编译器：扫描源代码，找到所有 EXEC SQL 开头的语句，把它们替换成对应的 DBMS 函数调用（库函数）。

2. C 编译器：编译替换后的纯 C 代码。

A 错：C 编译器不认识 EXEC SQL，这是 SQL 的关键字，不是 C 的。

B 正确：这就是预编译器干的事。

C 错：预编译不是可选的，是必须的。没有预编译，C 编译器拿到带 EXEC SQL 的代码直接报错。

D 错：预编译器也会处理 DECLARE SECTION 中的主变量声明，因为需要知道哪些变量是主变量、什么类型，才能生成正确的函数调用。

预编译器就像翻译官——C 编译器只懂 C 语言，EXEC SQL 是外语，翻译官先把外语翻成 C 能懂的库函数调用，C 编译器再接手编译。



第 10 题

答案：B

解析：

题目要求：逐行读取 → 判断 → 决定是否更新当前行。

A 错：一次性全部取出来在 C 里处理，如果数据量大，内存扛不住。而且你取出来了再更新，怎么定位到具体哪一行？还得再查一遍。

B 正确：游标逐行 FETCH，用 WHERE CURRENT OF 更新当前行，一次定位，一次更新，高效且精准。

C 错：一次性更新所有行？题目说的是"根据业务逻辑决定是否更新该行"，不是无脑全更新。

D 错：每行单独执行 SELECT + UPDATE，等于每行要访问数据库两次，效率极低。游标方案只需要一次查询打开游标，逐行判断更新。

游标 + WHERE CURRENT OF 是"边看边改"的最佳方案：像在 Excel 里逐行浏览，看到需要改的就当场改掉，不需要的就跳过。不用每看一行就重新打开一次表格。

答案汇总

1.B 2.A 3.B 4.C 5.D 6.B 7.B 8.A 9.B 10.B

